SMART MOVIE RECOMMENDATION SYSTEM

CodeVedX Internship Project

Submitted By

Name: Aarti Prajapati
Enrollment Number: 21231258
Course: B.Tech CSE (AI & ML), 7th Semester
College: IIMT University, Meerut ,Uttar Pradesh
Email : aartiaiml3009@gmail.com

Project Theme

Artificial Intelligence & Machine Learning (AI/ML)
Sub-Domain: Recommendation System
Project Type: Content-Based Movie Recommendation System

GitHub Repository

https://github.com/aartiprajapati-ai/CODEVEDX/tree/main/Smart_Recommendation_System

---

Academic Year: 2026–2027

INTRODUCTION

The **Smart Movie Recommendation System** is a Machine Learning-based application developed to recommend movies based on their content and similarity. With the rapid growth of online streaming platforms, users often face difficulty in finding movies that match their interests. A recommendation system helps solve this problem by suggesting relevant movies quickly and efficiently.

This project uses a **Content-Based Filtering** approach, where important movie features such as **genres, keywords, cast, crew, and overview** are analyzed to determine the similarity between movies. The textual information is converted into numerical vectors using **CountVectorizer**, and **Cosine Similarity** is used to identify movies with similar characteristics.

A simple and interactive **Streamlit** web application was developed, allowing users to select a movie from a list and instantly receive recommendations for similar movies. This project demonstrates the practical implementation of **Machine Learning**, **feature engineering**, **text processing**, and **recommendation algorithms** in solving a real-world problem.

The main objective of this project is to build an efficient, user-friendly, and accurate movie recommendation system that enhances the user experience by providing personalized movie suggestions.

PROBLEM STATEMENT

In many organizations, employees frequently ask similar questions regarding office timings, leave policies, attendance, password resets, salary slips, and other HR or IT-related services. Handling these repetitive queries manually requires significant time and effort from the support team, leading to increased workload, slower response times, and reduced operational efficiency.

As the number of employees grows, providing quick and consistent responses to every query becomes more challenging. Delays in resolving common issues can negatively affect employee productivity and overall workplace experience.

To address this challenge, there is a need for an intelligent and automated system that can instantly answer frequently asked questions without requiring continuous human intervention. An AI-powered chatbot can provide accurate, fast, and reliable responses, improving employee satisfaction while reducing the workload of HR and IT support teams.

The **AI Chatbot for Internal Helpdesk** is developed to solve this problem by using **Natural Language Processing (NLP)** and **Machine Learning** techniques to understand employee queries and deliver relevant responses through a simple Flask-based web application

PROJECT OBJECTIVE

The main objectives of the **Smart Movie Recommendation System** are:

- To develop a **Machine Learning-based movie recommendation system** that suggests similar movies based on their content.
- To implement a **Content-Based Filtering** approach for generating personalized movie recommendations.
- To preprocess and analyze movie data using **Python** and **Pandas**.
- To extract meaningful features such as **genres, keywords, cast, crew, and overview** for recommendation.
- To calculate movie similarity using **CountVectorizer** and **Cosine Similarity**.
- To build a simple and interactive **Streamlit web application** for users to receive movie recommendations.
- To provide fast, accurate, and relevant movie suggestions that improve the user experience.
- To gain practical knowledge of **Machine Learning**, **Recommendation Systems**, **Feature Engineering**, and **Web Application Development**.

EXPECTED OUTCOMES

The expected outcomes of the **Smart Movie Recommendation System** are:

- To develop a **Machine Learning-based recommendation system** that suggests movies based on content similarity.
- To provide users with **accurate and relevant movie recommendations** according to their selected movie.
- To implement **Content-Based Filtering** using movie features such as genres, keywords, cast, crew, and overview.
- To calculate movie similarity efficiently using **CountVectorizer** and **Cosine Similarity**.
- To build a **user-friendly Streamlit web application** for easy interaction and instant recommendations.
- To improve the user's movie discovery experience by reducing the time required to search for similar movies.
- To enhance practical knowledge of **Machine Learning**, **Feature Engineering**, and **Recommendation Systems**.
- To create a scalable project that can be extended with advanced recommendation techniques and real-time movie databases in the future.

SCOPE OF THE PROJECT

The **Smart Movie Recommendation System** is designed to recommend movies based on their content using **Machine Learning** techniques. The system analyzes movie features such as **genres, keywords, cast, crew, and overview** to identify similarities and suggest relevant movies to users.

The project focuses on implementing a **Content-Based Filtering** approach, which provides personalized movie recommendations based on the selected movie. A simple and interactive **Streamlit** web application enables users to select a movie and instantly receive recommendations.

This project can be further enhanced by integrating **TMDB APIs** to display movie posters, ratings, trailers, and detailed information. It can also be improved by implementing **Collaborative Filtering**, **Hybrid Recommendation Systems**, and **Deep Learning** techniques to provide more personalized and accurate recommendations. Additionally, the system can be deployed on cloud platforms and integrated into online streaming services to improve the overall user experience.

Overall, this project demonstrates the practical application of **Artificial Intelligence**, **Machine Learning**, and **Recommendation Systems** in solving real-world content recommendation problems.

LIMITATIONS

The **Smart Movie Recommendation System** has the following limitations:

- The system uses **Content-Based Filtering**, so recommendations are based only on movie features and not on user preferences or ratings.
- It recommends movies only from the **TMDB 5000 Movies Dataset** and cannot suggest newly released movies unless the dataset is updated.
- The quality of recommendations depends on the accuracy and completeness of the dataset.
- The system does not consider **user watch history**, likes, or viewing behavior for personalized recommendations.
- It does not provide **real-time updates**, as the recommendation model is trained on a static dataset.
- Movies with limited or missing information may receive less accurate recommendations.
- The current system does not display additional details such as **movie posters, trailers, reviews, or ratings**.
- Advanced recommendation techniques such as **Collaborative Filtering**, **Hybrid Recommendation Systems**, and **Deep Learning models** are not implemented in the current version.

SOFTWARE REQUIREMENTS

The following software and tools were used to develop the **Smart Movie Recommendation System**:

Software/Tool	Purpose
Operating System	Windows 10/11 or Linux
Programming Language	Python 3.x
IDE/Code Editor	Visual Studio Code (VS Code)
Notebook Environment	Google Colab / Jupyter Notebook
Framework	Streamlit
Libraries	Pandas, NumPy, Scikit-learn, Pickle
Version Control	Git & GitHub
Dataset Format	CSV Files (TMDB 5000 Movies Dataset)

HARDWARE REQUIREMENTS

The following hardware configuration is sufficient to develop and run the **Smart Movie Recommendation System**:

Hardware Component	Minimum Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or above
RAM	4 GB (8 GB Recommended)
Storage	Minimum 5 GB Free Disk Space
Display	1366 × 768 Resolution or Higher
Internet Connection	Required for downloading the dataset, installing libraries, and accessing GitHub

PYTHON LIBRARIES USED

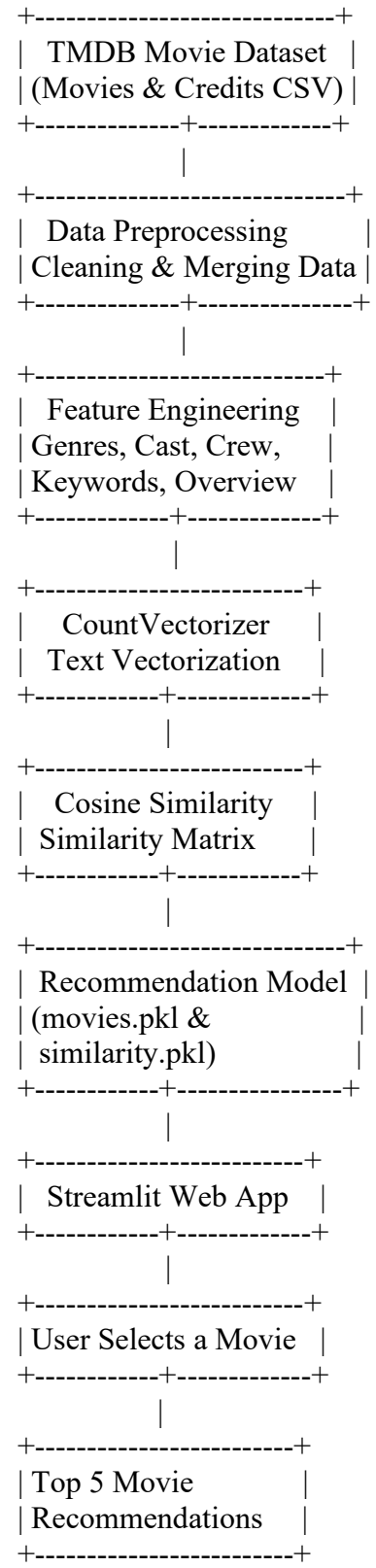
- **Pandas** – Data loading, cleaning, and preprocessing.
- **NumPy** – Numerical computations and array operations.
- **Scikit-learn** – Used for **CountVectorizer** and **Cosine Similarity**.
- **Pickle** – Saving and loading the trained recommendation model.
- **Streamlit** – Developing the interactive web application.

SYSTEM ARCHITECTURE

The **Smart Movie Recommendation System** follows a Content-Based Filtering approach to recommend movies based on their similarity. The system processes movie data through multiple stages, including data preprocessing, feature engineering, vectorization, similarity calculation,

and recommendation generation.

System Architecture Flow



WORKING OF THE SYSTEM

The **Smart Movie Recommendation System** works by analyzing the content of movies and recommending similar movies based on their features. It uses a **Content-Based Filtering** approach along with **Machine Learning** techniques to generate personalized recommendations.

Step 1: Data Collection

- The system uses the **TMDB 5000 Movies Dataset**, which consists of two CSV files:
  - **tmdb_5000_movies.csv**
  - **tmdb_5000_credits.csv**
- These files contain information such as movie titles, genres, cast, crew, keywords, and overviews.

Step 2: Data Preprocessing

- The movie and credits datasets are merged into a single dataset.
- Missing values are removed, and only the required columns are selected.
- The data is cleaned and prepared for further processing.

Step 3: Feature Engineering

- Important movie features such as **genres, keywords, cast, crew, and overview** are extracted.
- These features are combined into a single **tags** column, which represents the content of each movie.

Step 4: Text Vectorization

- The **CountVectorizer** converts the text in the **tags** column into numerical feature vectors.
- This allows the system to compare movies mathematically.

Step 5: Similarity Calculation

- **Cosine Similarity** is used to calculate the similarity score between all movies.
- Movies with higher similarity scores are considered more relevant to each other.

Step 6: Model Saving

- The processed movie dataset and similarity matrix are saved as **movies.pkl** and **similarity.pkl** using the Pickle library.
- These files are later used by the web application without retraining the model.

Step 7: User Input

- The user selects a movie from the dropdown list in the **Streamlit** web application.
- The selected movie is passed to the recommendation function.

Step 8: Recommendation Generation

- The system searches for the selected movie in the dataset.
- It retrieves the similarity scores and identifies the **Top 5 most similar movies**.
- The recommended movie titles are displayed to the user instantly.

DATASET DESCRIPTION

The **Smart Movie Recommendation System** uses the **TMDB 5000 Movies Dataset**, a publicly available dataset that contains detailed information about movies. This dataset is widely used for developing and evaluating movie recommendation systems.

The project uses the following two CSV files:

- **tmdb_5000_movies.csv** – Contains movie details such as title, genres, overview, keywords, popularity, ratings, and release date.
- **tmdb_5000_credits.csv** – Contains information about the cast and crew of each movie.

Both datasets were merged using the **movie_id** to create a unified dataset for preprocessing and recommendation generation.

Dataset Features

Column Name	Description
movie_id	Unique identifier for each movie
title	Name of the movie
overview	Brief description or storyline of the movie
genres	Categories of the movie (Action, Comedy, Drama, etc.)
keywords	Important keywords describing the movie
cast	Main actors appearing in the movie
crew	Director and other crew members

DATA PREPROCESSING

Before generating recommendations, the dataset was preprocessed using the following steps:

- Merged the **Movies** and **Credits** datasets.
- Removed missing and unnecessary values.
- Selected important columns such as **movie_id, title, overview, genres, keywords, cast, and crew**.
- Converted JSON-like data into Python lists using **ast.literal_eval()**.
- Extracted useful information from the **genres, keywords, cast, and crew** columns.
- Combined all selected features into a single **tags** column.
- Applied text preprocessing to prepare the data for vectorization.
- Converted the processed text into numerical vectors using **CountVectorizer**.

METHODOLOGY

The **Smart Movie Recommendation System** was developed using a systematic Machine Learning methodology to recommend movies based on their content. The project follows a sequence of data collection, preprocessing, feature engineering, vectorization, similarity calculation, model generation, and deployment.

Step 1: Data Collection

- The **TMDB 5000 Movies Dataset** was used for this project.
- Two CSV files, **tmdb_5000_movies.csv** and **tmdb_5000_credits.csv**, were collected.
- These files contain detailed information about movies, including genres, cast, crew, keywords, and overviews.

Step 2: Data Preprocessing

- The movies and credits datasets were merged into a single dataset using the **movie_id**.
- Missing values and unnecessary columns were removed.
- Important columns such as **title, genres, keywords, cast, crew, and overview** were selected.

- JSON-like data was converted into Python objects using **ast.literal_eval()**.

Step 3: Feature Engineering

- Relevant information was extracted from the **genres, keywords, cast, and crew** columns.
- These features were combined with the movie overview to create a single **tags** column representing each movie.
- Text preprocessing was performed to make the data suitable for recommendation.

Step 4: Text Vectorization

- The **CountVectorizer** technique was applied to convert the textual **tags** into numerical feature vectors.
- This enabled the system to mathematically compare movie content.

Step 5: Similarity Calculation

- **Cosine Similarity** was used to calculate the similarity score between movies based on their feature vectors.
- Movies with higher similarity scores were considered more closely related.

Step 6: Model Saving

- The processed movie dataset and similarity matrix were saved as **movies.pkl** and **similarity.pkl** using the **Pickle** library.
- These files allow the recommendation system to generate recommendations without repeating the training process.

Step 7: Deployment

- A **Streamlit** web application was developed to provide a user-friendly interface.
- Users can select a movie from a dropdown list.
- The system processes the selected movie and displays the **Top 5 most similar movie recommendations** instantly.

PROJECT IMPLEMENTATION

The **Smart Movie Recommendation System** was implemented using **Python** and **Machine Learning** techniques to recommend movies based on their content similarity. The implementation followed a structured workflow, beginning with data collection and preprocessing, followed by feature engineering, similarity calculation, model generation, and deployment through a Streamlit web application.

1. Data Collection

- The project uses the **TMDB 5000 Movies Dataset**, which consists of two CSV files:
  - **tmdb_5000_movies.csv**
  - **tmdb_5000_credits.csv**

- These datasets contain movie information such as title, genres, overview, keywords, cast, and crew.

2. Data Preprocessing

- The two datasets were merged using the **movie_id** column.
- Missing values and unnecessary columns were removed.
- Only the required attributes, including **title, genres, keywords, cast, crew, and overview**, were selected for recommendation.

3. Feature Engineering

- JSON-formatted columns were converted into Python objects using **ast.literal_eval()**.
- Important information such as genres, keywords, top cast members, director, and overview was extracted.
- All extracted features were combined into a single **tags** column representing each movie.

4. Text Vectorization

- The **CountVectorizer** technique was used to convert the textual **tags** into numerical feature vectors.
- This transformation enabled the system to compare movies based on their content.

5. Similarity Calculation

- **Cosine Similarity** was applied to calculate similarity scores between all movies.
- The similarity matrix was generated, allowing the system to identify movies with similar content.

6. Model Saving

- The processed movie dataset and similarity matrix were saved using the **Pickle** library.
- The generated files:
  - **movies.pkl**
  - **similarity.pkl**
- These files allow the recommendation system to provide results without rebuilding the model each time.

7. Streamlit Web Application

- A **Streamlit** application was developed to provide an interactive user interface.
- Users can select a movie from the dropdown list.
- When the **Recommend** button is clicked, the system calculates similarity scores and displays the **Top 5 recommended movies**.

.

RESULTS AND OUTPUT

The **Smart Movie Recommendation System** was successfully developed and tested using **Machine Learning** techniques. The system uses a **Content-Based Filtering** approach to recommend movies based on their similarity. By analyzing features such as **genres, keywords, cast, crew, and overview**, the application generates relevant movie recommendations for the selected movie.

The application was deployed using **Streamlit**, providing a simple and interactive interface where users can select a movie from a dropdown list and instantly receive recommendations. The recommendation engine efficiently identifies similar movies using **CountVectorizer** and **Cosine Similarity**, delivering fast and accurate results.

Project Results

- Successfully implemented a **Content-Based Movie Recommendation System**.

- Generated the **Top 5 similar movie recommendations** based on the selected movie.
- Applied **CountVectorizer** for text vectorization and **Cosine Similarity** for similarity calculation.
- Developed an interactive **Streamlit web application** for user-friendly recommendations.
- Stored the processed movie dataset and similarity matrix using **Pickle (.pkl)** files for efficient reuse.
- Delivered relevant movie recommendations with quick response time.

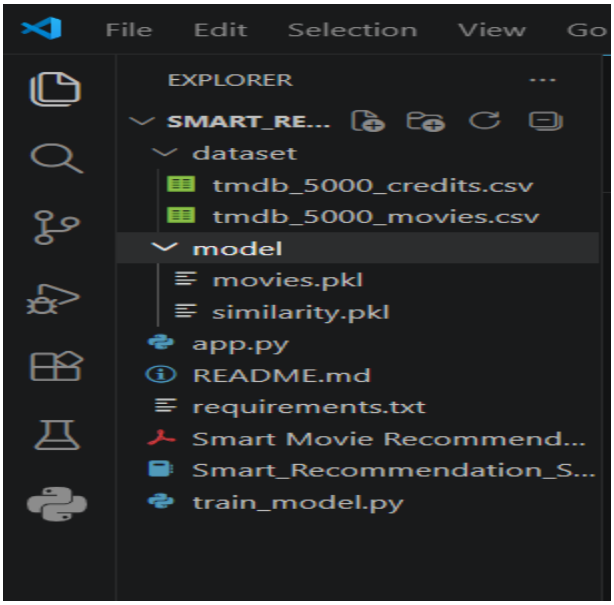
Output Screenshots

1. Project Folder Structure

Description:

The project is organized into separate folders for the dataset, trained model files, source code, documentation, and notebook, ensuring a clean and maintainable project structure

Screenshot:

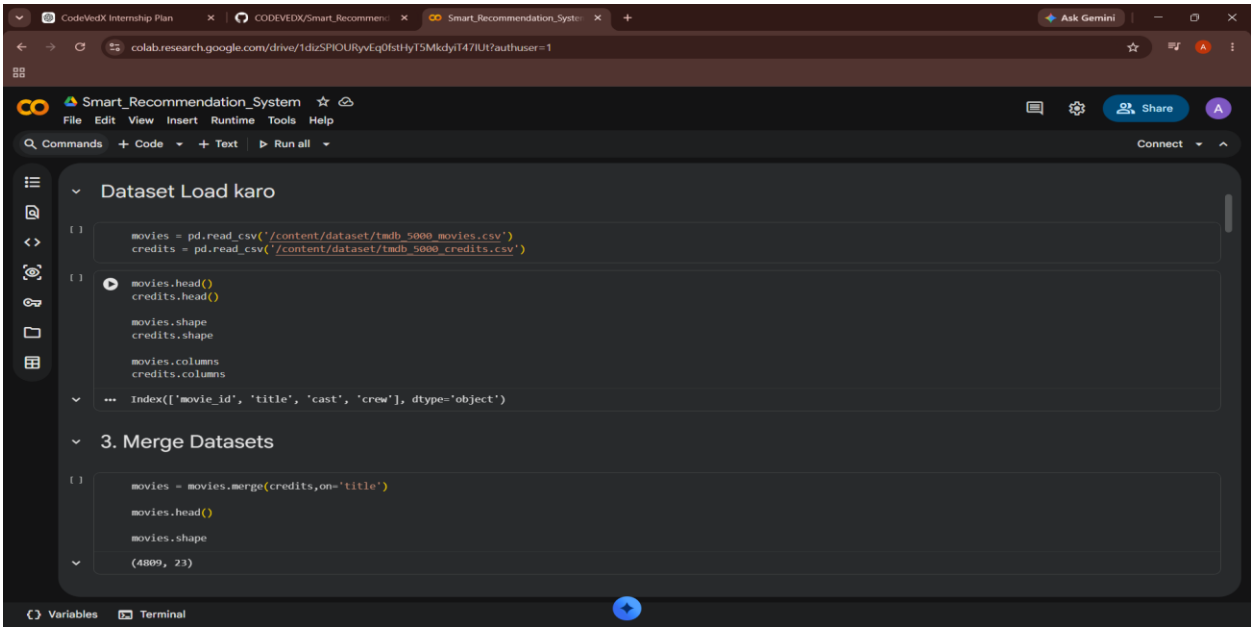


2. Dataset Loading

Description:

The TMDB 5000 Movies Dataset was successfully loaded into the project for preprocessing and recommendation model development.

Screenshot:

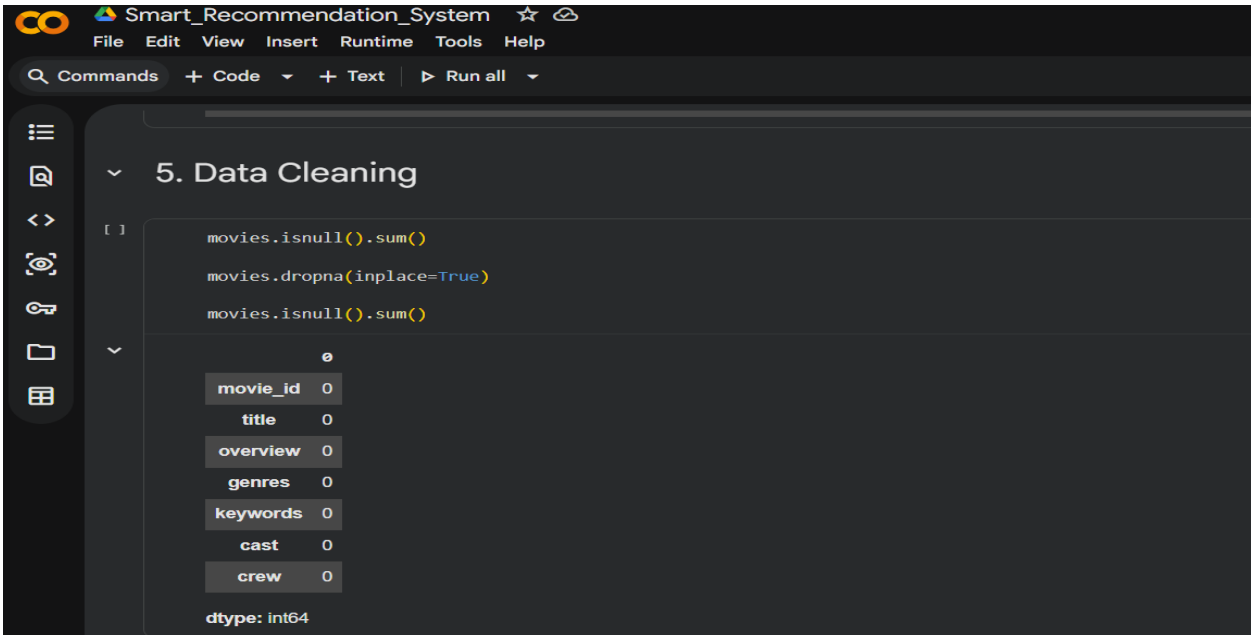


3. Data Preprocessing

Description:

The dataset was cleaned by removing missing values, selecting relevant columns, and preparing the movie information for feature engineering.

Screenshot:

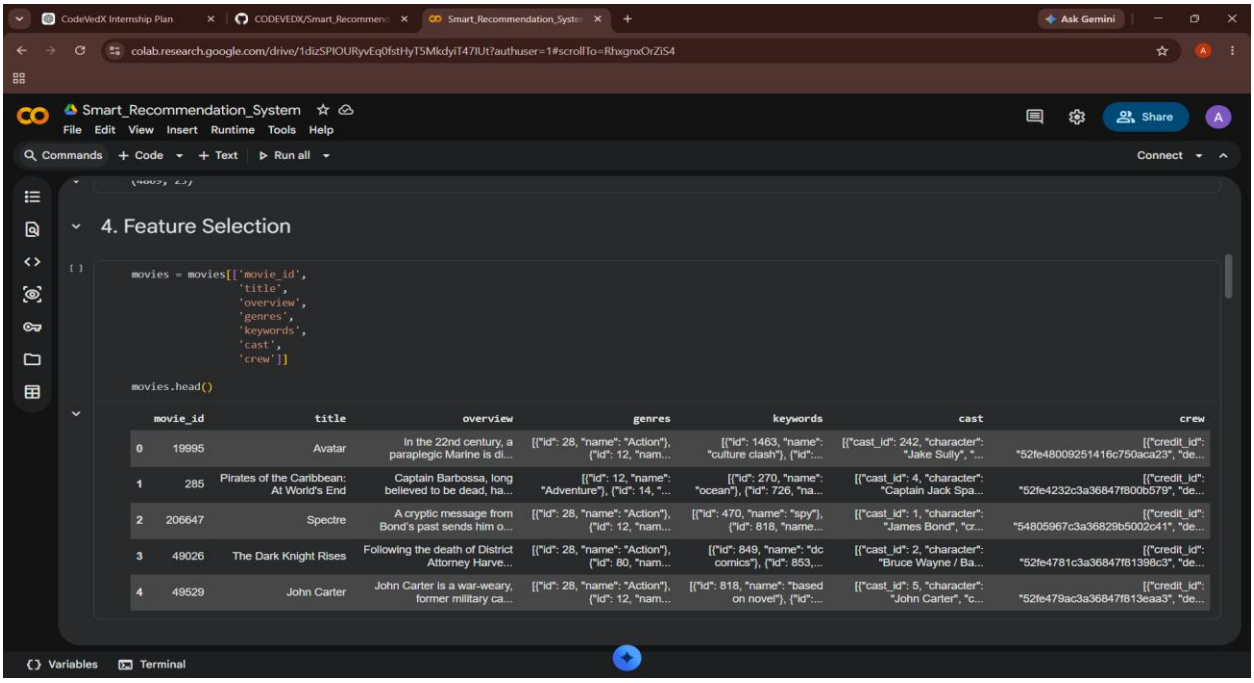


4. Feature Engineering

Description:

Important movie features such as genres, keywords, cast, crew, and overview were combined into a single **tags** column for content-based recommendation.

Screenshot:

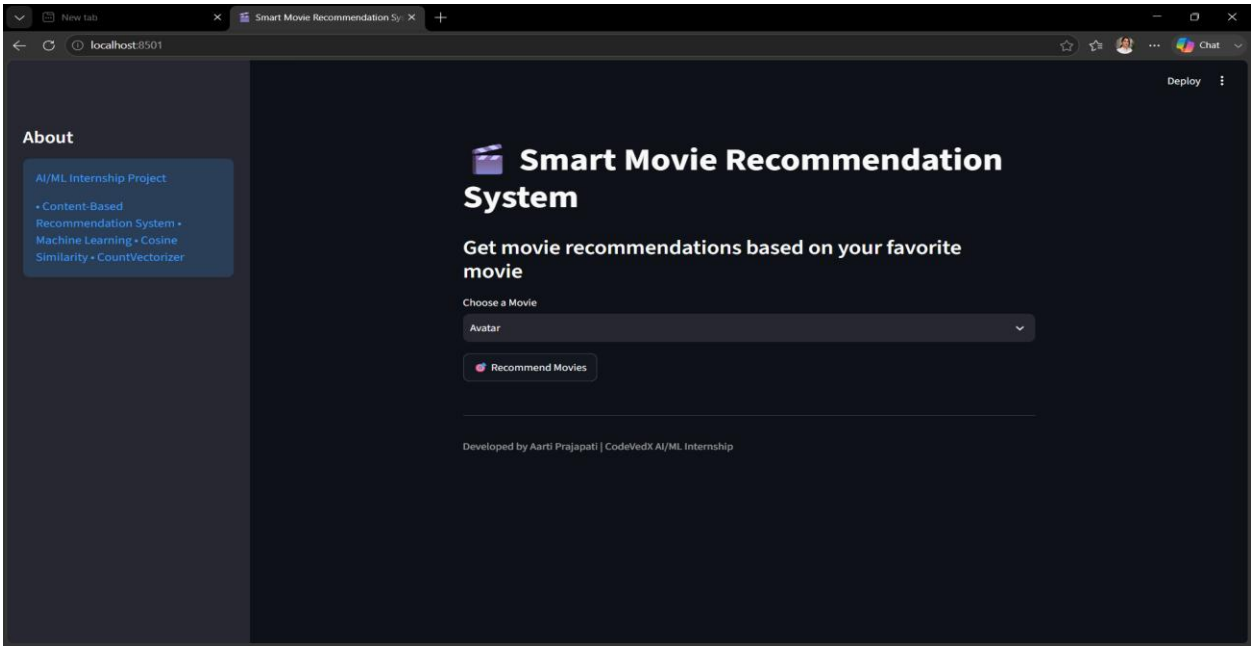


5. Streamlit Home Page

Description:

The Streamlit application provides an interactive interface where users can select a movie and request recommendations.

Screenshot:

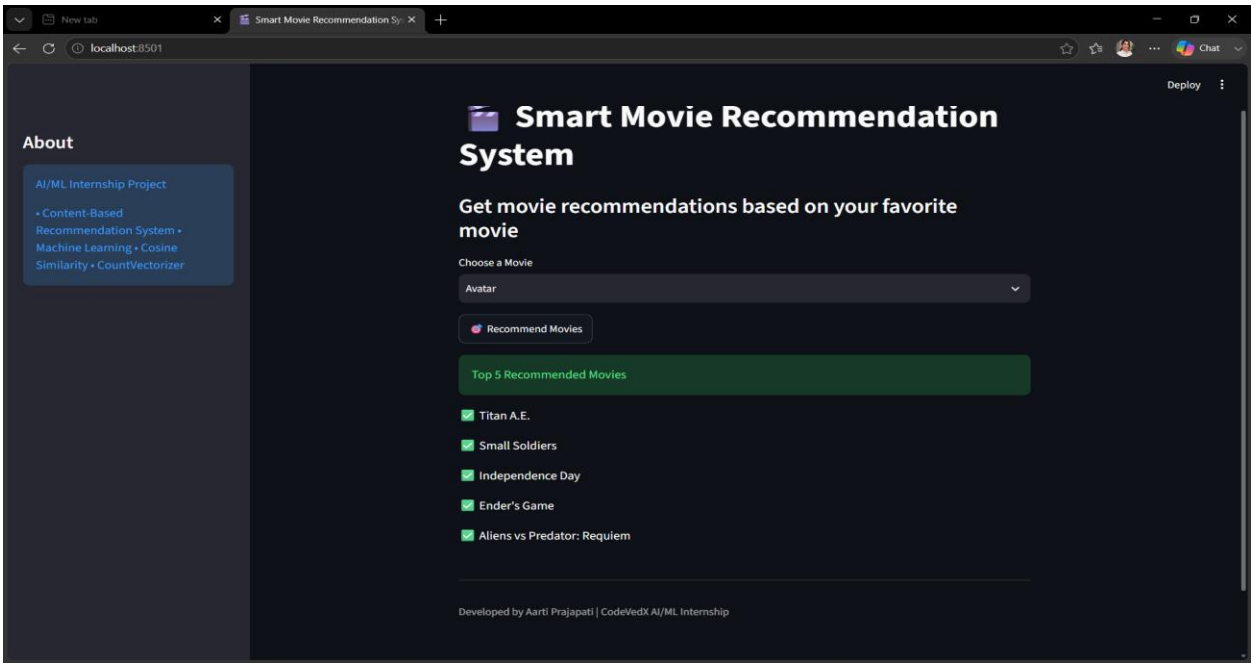


6. Recommendation Output

Description:

After selecting a movie, the recommendation system analyzes the similarity scores and displays the top five most similar movies.

Screenshot:



ADVANTAGES

The **Smart Movie Recommendation System** offers several benefits to users by providing personalized and efficient movie suggestions. The key advantages are:

- Provides **personalized movie recommendations** based on content similarity.
- Helps users **discover movies** that match their interests quickly.
- Reduces the time required to search for relevant movies.
- Uses **Content-Based Filtering**, making recommendations more relevant to the selected movie.
- Delivers **fast and accurate recommendations** using **CountVectorizer** and **Cosine Similarity**.
- Features a **simple and user-friendly Streamlit interface** for easy interaction.
- Can be easily updated with new movie datasets and additional features.
- Improves practical understanding of **Machine Learning**, **Feature Engineering**, and **Recommendation Systems**.
- Can be extended to include **movie posters, ratings, trailers, and user reviews**.
- Serves as a strong foundation for building advanced recommendation systems used in streaming platforms such as Netflix and Amazon Prime Video.

CONCLUSION

The **Smart Movie Recommendation System** was successfully developed using **Machine Learning** techniques and a **Content-Based Filtering** approach. The system effectively recommends movies by analyzing features such as **genres, keywords, cast, crew, and**

overview. Using **CountVectorizer** and **Cosine Similarity**, it identifies movies with similar content and provides accurate recommendations.

The project enhanced practical knowledge of **data preprocessing, feature engineering, recommendation algorithms, and Streamlit application development**. It demonstrates how Machine Learning can be applied to solve real-world problems by improving the movie discovery experience for users.

Overall, the project achieved its objectives by providing a **simple, efficient, and user-friendly recommendation system**. It also serves as a strong foundation for developing more advanced recommendation systems with features such as **user preferences, collaborative filtering, hybrid models, and real-time movie data** in the future.

FUTURE SCOPE

The **Smart Movie Recommendation System** can be further enhanced by integrating advanced technologies and additional features to improve its accuracy, functionality, and user experience.

- Implement **Collaborative Filtering** to recommend movies based on user ratings and preferences.
- Develop a **Hybrid Recommendation System** by combining Content-Based and Collaborative Filtering techniques.
- Integrate the **TMDB API** to display movie posters, ratings, trailers, release dates, and other real-time information.
- Use **Deep Learning** and advanced recommendation algorithms to improve recommendation accuracy.
- Add **user login, watch history, and personalized profiles** for customized recommendations.
- Include **user ratings and feedback** to continuously improve recommendation quality.
- Expand the system by using larger and regularly updated movie datasets.
- Deploy the application on **cloud platforms** such as AWS, Azure, or Google Cloud for better scalability and accessibility.
- Develop **mobile and desktop applications** to make the recommendation system available across multiple platforms.
- Extend the system to recommend **TV shows, web series, documentaries, and other entertainment content**, making it a complete entertainment recommendation platform.

Overall, the project provides a strong foundation for developing intelligent and scalable recommendation systems that can be used in real-world streaming platforms and online entertainment services.

REFERENCES

The following resources were referred to during the development of this project:

1. TMDB 5000 Movies Dataset (Kaggle)
2. Scikit-learn Documentation
3. Pandas Documentation
4. NumPy Documentation
5. Streamlit Documentation
6. Python Official Documentation
7. GitHub Documentation
8. Visual Studio Code Documentation

APPENDIX

Appendix A – Project Folder Structure

```
Smart_Recommendation_System
├── dataset
│   ├── tmdb_5000_movies.csv
│   └── tmdb_5000_credits.csv
├── model
│   ├── movies.pkl
│   └── similarity.pkl
├── app.py
├── train_model.py
├── requirements.txt
├── README.md
└── Smart_Recommendation_System.ipynb
```

Appendix B – Python Libraries Used

The following Python libraries were used in the project:

- Pandas
- NumPy
- Scikit-learn
- Streamlit
- Pickle
- AST (Abstract Syntax Trees)

Appendix C: Project Files

The project consists of the following files:

- **tmdb_5000_movies.csv** – Movie dataset
- **tmdb_5000_credits.csv** – Movie credits dataset
- **train_model.py** – Recommendation model training script
- **app.py** – Streamlit web application
- **movies.pkl** – Processed movie dataset
- **similarity.pkl** – Similarity matrix
- **requirements.txt** – Python dependencies
- **README.md** – Project documentation
- **Smart_Recommendation_System.ipynb** – Jupyter Notebook containing data preprocessing, feature engineering, and model development

Appendix D: Screenshots

The following screenshots are included in the report:

- Project Folder Structure
- Dataset Loading
- Data Preprocessing
- Feature Engineering
- Streamlit Home Page
- Movie Recommendation Output

Appendix E: Acronyms

Abbreviation	Meaning
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
CSV	Comma-Separated Values
API	Application Programming Interface
UI	User Interface
IDE	Integrated Development Environment
TMDB	The Movie Database